Clean Air Finder

A local, portable Windows application answering: **Where is the cleanest measured air in Bulgaria, now or during a chosen historical period?**

The default view names official Bulgarian monitoring stations and their site addresses. Leave both dates blank for the latest common reported hour; choose dates for history. The original city-average view is available in **Search settings**.

Source: Clean-Air-Repository on GitHub

The repository contains the complete project source, portable runtime, installed Python packages and measurement database snapshot. Large binaries and the database use **Git LFS**. The original development milestones are retained locally on a backup branch rather than included in the public branch's history.

The interface, API, SQLite database, Python runtime, Python packages, monitoring, logs and archives all live in this folder. There is no Docker, Node.js, database server, cloud account or application login.

Start the complete application

1. Obtain the complete Git checkout in a **writable local folder** on 64-bit Windows 10 or 11. If using a handover ZIP, extract it first. Do not run the application inside a ZIP, on a network drive or in a folder synchronized while the database is open.
2. Double-click `Start-CleanAir.cmd`, or run:

```
.\Start-CleanAir.cmd
```

3. Your browser opens at **http://127.0.0.1:8000**.
4. Keep the console open while using the app. To stop, double-click **Stop-CleanAir.cmd**, or press **Ctrl+C** in the console.

The complete checkout and handover include an already imported real-data database. With a fresh, empty database, the UI displays the initial backfill progress and does not invent a winner before the complete import has succeeded.

Internet is needed for initial collection and updates, **not for searches or the interface**. To deliberately run without contacting the source:

```
.\Start-CleanAir.cmd --offline
```

Use `--no-browser` when starting from an automated check or when you prefer to open the local address yourself.

Prometheus is included and starts automatically at **http://127.0.0.1:9090**. No Grafana account is needed. Corporate device policies may require IT approval for portable executables; the application does not change security settings or require administrator privileges.

Make a handover ZIP

First double-click **Stop-CleanAir.cmd** and wait for it to finish. It requests a clean shutdown, stops leftover project processes, flushes SQLite and checks the important file locks. It does not stop unrelated Python programs. Running it again when the project is already stopped is safe.

Then double-click `Prepare-Handover.cmd`, or run:

```
.\Prepare-Handover.cmd
```

For the smaller ready-to-run package without local Git/LFS object caches:

```
.\Prepare-Handover.cmd --without-git
```

This creates:

```
dist\CleanAirFinder-Windows-x64.zip
dist\CleanAirFinder-Windows-x64.zip.sha256
```

The ZIP contains the source, configuration, bundled runtimes and licenses, automated tests, documentation, archives and a **verified SQLite snapshot**. Local handover packages include Git history when packaged from a Git checkout; `--without-git` and public release ZIPs omit internal Git metadata. Temporary downloads, test output, logs and monitoring history are excluded. Recipients extract it and use the same startup command. They do not need Python installed.

You can also zip the workspace yourself **after stopping the application**. Exclude `dist` to avoid placing an existing ZIP inside another ZIP. Do not copy only an open SQLite database file while omitting its WAL.

What the result means

- The normal answer says **Cleanest measured station/city**. It measures pollution concentration: **lower pollution means cleaner air**, not a lower quality score.
- Leave **both dates blank** for the latest comparable reported hour. All compared locations must report for that same hour, within the configured six-hour maximum age. No fresh comparison means an explicit error, not a stale "current" winner.
- Choose both inclusive dates for history; the same date means one completed day.
- Choose **PM2.5** or **PM10** explicitly. The new configured default is PM10, which has broader official Bulgarian station coverage; it is never silently substituted when a different pollutant was selected.
- Dates mean **UTC calendar days**, not Bulgarian local days. This preserves matching windows between EEA validated daily values and current hourly data.

- Normally the winner has the lowest average concentration for the selected time.
- A qualifying physical station receives equal daily weight; valid days receive equal period weight. Multiple sampling methods do not duplicate a station.
- The default station/period coverage requirements are 75% of expected hourly observations and 90% of requested days. Valid provider-reported daily values use provider validity information; unspecified hourly coverage stays unknown.
- Missing measurements never become zero. Excluded locations and reasons are shown.
- WHO 2021 daily references are **15 ug/m3 for PM2.5** and **45 ug/m3 for PM10**. Daily references are **not applied to hourly readings**.
- **Search settings** keeps advanced controls out of the main flow:
  - station details or the original city-average view;
  - lowest concentration, most valid days within the daily reference, or greatest improvement versus earlier stored history;
  - cleanest day or hour inside selected dates, optionally restricted to one location.
- Reference-day and improvement rankings need dates and are labelled as different questions. An improvement winner need not have the lowest absolute pollution. The baseline uses at least 30 earlier valid days at each contributing station.
- Cleanest-day/hour searches use the lowest concentration and the selected period's qualifying station cohort. Hour searches require actual hourly data; old daily averages cannot reconstruct it. The best 50 time combinations are shown with the total disclosed.
- A station address identifies the monitor, **not every nearby street**. The lowest measured concentration does not guarantee healthy air. WHO daily guidance uses an annual 99th-percentile convention; this is not compliance certification.

See methodology and limitations for the exact calculation.

Architecture

```
Browser
  |
  +-- FastAPI: HTML templates, local JSON API, health checks, metrics
        |
        +-- SQLite: source observations, indexed station-day aggregates,
        |           official station sites, city mappings, checkpoints
        |
        +-- One background collection worker
              +-- Public EEA metadata and Parquet downloads
              +-- Initial backfill, conditional updates and verified archiving

Bundled Prometheus --> local /metrics --> local monitoring storage and alerts
```

One application process and one collection worker are intentional. SQLite WAL allows readers alongside a short-lived writer. This is a single-machine, localhost deployment, not a distributed service. Searches never call the EEA. Indexing and daily rollups keep query work independent of raw hourly volume.

Workspace layout

Folder	Purpose
`clean_air`	Python application, templates and static assets
`config`	Validated settings, reference profiles, Prometheus configuration and alerts
`runtime`	Portable Python 3.13, installed packages, Prometheus and component manifest
`tests`	Offline calculation, import, archive, API and failure-behaviour tests
`tools`	Reproducible runtime builder and complete HTTP benchmark
`docs`	Methodology, operations, attribution, AI usage and verification results
`var\data`	Persistent SQLite database
`var\archives`	Verified compressed records older than 36 months
`var\backups`	Consistent SQLite snapshots
`var\logs`	Bounded application logs and monitoring logs
`var\monitoring`	Prometheus time-series storage
`var\reports`	Measured benchmark results
`dist`	Generated handover ZIP and SHA-256 checksum

No project state is stored in your user profile. The bundled runtime does not replace or modify your existing Python installation.

Configuration

Edit `config\settings.toml` or `config\profiles.toml`, then restart. Invalid configuration stops startup with an explicit error.

Useful settings include ports, collection interval, 12-36 months of initial backfill, coverage requirements and the default pollutant. Active retention is 36 months. Calendar days are deliberately restricted to UTC. Set `collection.enabled = false` for persistent offline use.

Built-in profile edits must retain honest names, source attribution and version information. For personal preferences, select the UI's **custom** profile.

Test and measure

Run offline lint and automated tests:

```
.\Test-CleanAir.cmd
```

Measure the complete comparison service on the stored real dataset:

```
.\runtime\python\python.exe -m clean_air benchmark --iterations 100
```

With the application running, measure complete HTTP responses with concurrent clients and validate the successful output shape:

```
.\runtime\python\python.exe .\tools\benchmark_http.py --requests 100 --concurrency
4
```

The initial engineering target is **p95 below 300 ms**, measured separately for one-day, thirty-day and full-year queries for both pollutants. Benchmarks return a nonzero exit status if the target is missed; results are written to `var\reports`. This target depends on the documented machine and load.

Data source and attribution

The EEA Air Quality Download Service provides anonymous access to validated E1a and provisional E2a measurements. Both are collected locally; models, gap-filled estimates and historical Airbase data are not mixed into the ranking.

The national ИАОС map provides factual station names, site addresses and coordinates. Its 32 sites were uniquely matched to EEA station IDs at identical coordinates and stored in the versioned station catalog. On 18 October 2026 its measurement fields contained zero placeholders with no pollutant or timestamp, so those were **not** imported. The station/city selector is a change of comparison view, not two independent measurement sources.

See verified source research and limitations. We do not promise agreement with Google or an AI answer based on a different pollutant, station network, timestamp, model or ranking definition. No AI API is used.

The Air Quality e-Reporting dataset record identifies **CC BY 4.0, copyright European Commission**. Attribution and an indication of local filtering/aggregation appear in the interface, API and third-party notices.

PM2.5 is important for health but has substantially fewer eligible Bulgarian city stations than PM10. The application reports the actual comparison scope rather than claiming national completeness.

Using the complete Git checkout

Install Git and Git LFS before cloning so the actual portable binaries and database are downloaded instead of small pointer files:

```
git lfs install
git clone https://github.com/lemargina/Clean-Air-Repository.git
Set-Location '.\Clean-Air-Repository'
git lfs pull
.\Start-CleanAir.cmd
```

The runtime, its packages and the database snapshot are included in the repository. Logs, monitoring history, caches, backups and SQLite WAL/SHM files remain excluded. Git LFS has separate GitHub storage/bandwidth quotas.

GitHub-generated source archives may contain LFS pointers unless LFS objects are included in the repository's archive settings. For transfer without Git or Git LFS, create the optional handover ZIP described above.

Running collection updates the tracked database locally. Stop the application before committing or switching revisions; database changes are not automatically pushed to GitHub.

Rebuilding the bundled runtime

To recreate the portable runtime, use an installed Python 3.12+ on Windows x64:

```
python .\tools\build_portable.py
.\Start-CleanAir.cmd
```

The builder verifies official runtime SHA-256 checksums, installs target-specific wheels inside the workspace, uses the hash-pinned `requirements.lock`, and tests imports before replacing the active runtime. Build temporary files are local. Stop the app before rebuilding. A failed staging build is retained for diagnosis; inspect `var\build\runtime-staging` before removing that specific staging folder.

To intentionally resolve new package versions:

```
python .\tools\build_portable.py --refresh-dependencies
```

Review the updated lock and runtime manifest, then rerun tests, both benchmarks, and relocation checks. Do not assume a new dependency resolution is validated.

Further documentation

- Operations, recovery and archiving
- Methodology and trade-offs
- Third-party licenses and data attribution
- AI usage and corrections
- Verification results

With more time: independently validate station metadata and weighting with an air-quality specialist; add traffic/background station filtering; exercise larger multi-year histories; add external alert delivery; and consider PostgreSQL only if a multi-writer or multi-instance deployment becomes necessary.